

C++ Integer Types & Memory Limits

The Ultimate Quick Reference Guide for Competitive Programming

For platforms like LeetCode, Codeforces, AtCoder, and CSES, knowing the ranges of C++ integer types by heart is crucial. It allows you to confidently select the correct data type instantly without losing precious time during live contests.

1. Integer Types (Smallest → Largest)

Data Type	Size	Signed Range	Unsigned Range	Max Digits
<code>bool</code>	1 byte	0 or 1	-	1
<code>char</code>	1 byte	-128 to 127	0 to 255	3
<code>short</code>	2 bytes	-32,768 to 32,767	0 to 65,535	5
<code>int</code>	4 bytes	-2,147,483,648 to 2,147,483,647	0 to 4,294,967,295	10
<code>long</code> (Windows)	4 bytes	Same as int	Same	10
<code>long</code> (Linux/GCC)	8 bytes	$\pm 9 \times 10^{18}$	0 to 1.84×10^{19}	19
<code>long long</code>	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	0 to 18,446,744,073,709,551,615	19
<code>__int128</code> (GCC only)	16 bytes	$\pm 1.7 \times 10^{38}$	huge	38

2. Easy Contest Memory Table

Type	Safe Range
<code>int</code>	$\pm 2 \times 10^9$
<code>long long</code>	$\pm 9 \times 10^{18}$
<code>__int128</code>	$\pm 10^{38}$

* Note: Most competitive programmers remember only these three types for integer operations.

3. Decimal Digits Each Type Can Store

Type	Maximum Value	Digits
short	32,767	5
int	2,147,483,647	10
long long	9,223,372,036,854,775,807	19
__int128	$\approx 1.7 \times 10^{38}$	38

4. Powers of Two (Very Important)

Since computer memory is inherently binary, ranges align perfectly with powers of two:

Bits	Maximum Unsigned Value	Approx. Signed Value
8	$2^8 - 1 = 255$	± 128
16	65,535	$\pm 32K$
32	4.29×10^9	$\pm 2 \times 10^9$
64	1.84×10^{19}	$\pm 9 \times 10^{18}$
128	3.4×10^{38}	$\pm 1.7 \times 10^{38}$

5. Floating Point Types

Type	Size	Precision	Range
float	4 bytes	6–7 digits	$\pm 3.4 \times 10^{38}$
double	8 bytes	15–16 digits	$\pm 1.7 \times 10^{308}$
long double	16 bytes (GCC)	18–21 digits	very large

6. Which Type to Use Based on Value Size?

Upper Constraint Limit	Recommended Type
≤ 100	int
$\leq 10^5$	int
$\leq 10^6$	int
$\leq 10^7$	int
$\leq 10^8$	int
$\leq 10^9$	int
$\leq 2 \times 10^9$	int
$> 2 \times 10^9$	long long
$\leq 10^{18}$	long long
$> 10^{18}$	__int128 or Big Integer

7. Common Problem Constraints & Recommendations

Constraint Pattern in Problem	Recommended Type
Array Size $n \leq 10^5$	int
Element Value $a[i] \leq 10^9$	int
Sum of array elements $\leq 10^{14}$	long long
Product of values $\leq 10^{18}$	long long
Product of values $> 10^{18}$	__int128
Modulo arithmetic intermediate operations	long long (usually)
Prefix sums arrays	long long
DP table values	Usually long long
Distances/Edge weights in graphs	long long
Coordinates $\leq 10^9$	int
Area calculation ($x \times y$)	long long

8. Integer Limits Constants in C++

```
#include <climits>

INT_MIN      = -2147483648
INT_MAX      = 2147483647

LLONG_MIN    = -9223372036854775808
LLONG_MAX    = 9223372036854775807

SHRT_MIN     = -32768
SHRT_MAX     = 32767

CHAR_MIN     = -128
CHAR_MAX     = 127
```

9. When Does Overflow Happen?

Example with `int`

```
int x = 2000000000;
x += 1000000000;    // Overflow! Exceeds 2,147,483,647
```

Result: Signed integer overflow results in **undefined behavior** in C++.

Example with `long long`

```
long long x = 9000000000000000000LL;
x += 1000000000000000000LL;    // Overflow! Exceeds 9,223,372,036,854,775,807
```

Result: Similarly, signed overflow leads to undefined behavior.

10. Important Competitive Programming Rules

Rule 1: Cumulative Accumulation

If numbers can reach 10^9 individually, but you are summing many of them up, the total will exceed `int` capacity. Use `long long`.

Example: $n = 105$ elements, each $a[i] = 109 \rightarrow$ Total Sum = 10^{14} .

`int`



`long long`



Rule 2: Integer Promotion Pitfall

If you are multiplying two `int` variables, the multiplication is computed as an `int` expression **first**, before being assigned to the destination variable.

```
int a = 100000;
int b = 100000;
long long c = a * b;    // WRONG: Overflow occurs before assignment!
```

Correct Approach: Force type promotion by multiplying by `1LL`.

```
long long c = 1LL * a * b;    // CORRECT
```

Rule 3: Prefix Sum Arrays

Prefix sums accumulate changes across arrays over time and almost always require `long long` storage structures.

```
vector<long long> prefix(n + 1);
```

Rule 4: Graph Algorithmic Distances

Path lengths and cumulative distances in algorithms like Dijkstra, Floyd-Warshall, and Bellman-Ford should consistently use `long long`, as paths can rapidly outgrow standard integer limits.

Rule 5: Intermediate Modulo Evaluation

When computing answers under a modulo constraint (e.g., $10^9 + 7$ or `998,244,353`), intermediate multiplications can still easily exceed `int` before the modulo reduction is applied.

```
long long ans = (1LL * a * b) % MOD;
```

11. Contest Cheat Sheet (Memorize This Table)

Maximum Value Bound	Safest Data Type
10^2	int
10^3	int
10^4	int
10^5	int
10^6	int
10^7	int
10^8	int
10^9	int
10^{10}	long long
10^{12}	long long
10^{15}	long long
10^{18}	long long
10^{20}	__int128

12. What Most Competitive Programmers Actually Use

In practice, 95% of standard competitive programming problems reduce down to only these select workflows:

- **int** : Used for loops, indices, counts, and values up to approximately 2×10^9 .
- **long long** : Used for accumulations, large products, graph paths, dynamic programming tables, and bounds reaching up to 10^{18} .
- **double** : Leveraged for spatial geometry, probabilities, and exact floating-point precision metrics.
- **__int128** : Reserved for niche boundary tasks where scalar multiplications cross past 10^{18} values without a continuous modulo constraint.

Pro-tip: If you are unsure during a high-stakes contest, defaulting to **long long** for general numeric calculations is mathematically safer, provided memory restrictions are not overly severe.

Deep Dive: Detailed Breakdown of Table Architecture

Understand the math behind the columns to remove memorization permanently

Let's unpack every column of the primary architecture layout to see how values are derived from raw binary configurations.

Reference Metrics Base Table

Data Type	Size	Signed Range	Unsigned Range	Maximum Digits
char	1 byte	-128 to 127	0 to 255	3
short	2 bytes	-32,768 to 32,767	0 to 65,535	5
int	4 bytes	-2,147,483,648 to 2,147,483,647	0 to 4,294,967,295	10
long long	8 bytes	-9.22×10^{18} to 9.22×10^{18}	0 to 1.84×10^{19}	19

1. Data Type Column

Informs the underlying compiler exactly how much system memory space must be allocated to accommodate values. For instance:

```
int x = 10;    // Allocates 4 bytes of system memory
char c = 'A'; // Allocates exactly 1 byte of system memory
```

2. Size Column

Reflects physical memory usage metrics. The fundamental translation relies on: $1 \text{ ext{ Byte}} = 8 \text{ ext{ Bits}}$.

Size in Bytes	Total Binary Bits Available
1 byte	8 bits
2 bytes	16 bits
4 bytes	32 bits
8 bytes	64 bits
16 bytes	128 bits

An `int` consists of 4 bytes, equivalent to $4 \times 8 = 32$ individual binary digits. For example, the integer value `10` is represented under the hood as:

```
00000000 00000000 00000000 00001010
```

3. Signed Range Column

Specifies the strict geometric bounds (minimum and maximum values) when dealing with mixed sign formats (negative, zero, and positive inputs).

Taking `int` as our case study: it features 32 bits, allowing for $2^{32} = 4,294,967,296$ unique permutations. Because signed variables track negative spaces alongside positives, the pool splits evenly: half to the negative quadrant and half to non-negative boundaries.

Two's Complement Range Formula (for n bits):

$$-2^{n-1} \text{ to } 2^{n-1} - 1$$

For a 32-bit `int`, this computes directly as:

Minimum: $-2^{31} = -2,147,483,648$

Maximum: $2^{31} - 1 = 2,147,483,647$

Asymmetry Note: The negative side spans one value further because the positive representation includes zero, shifting the positive ceiling downwards by exactly 1 unit.

4. Unsigned Range Column

Unsigned means that negative numeric states are disabled entirely. The most significant bit no longer acts as a sign indicator; instead, all bits are dedicated explicitly to absolute scalar values.

For an `unsigned int` with 32 operational bits, the maximum scale bounds span from `0` up to $2^{32} - 1$, mapping directly to `4,294,967,295`.

Signed Visual Layout:

Negative Boundary $\leftarrow [-2,147,483,648]$ — (0) — $[+2,147,483,647]$ $\rightarrow$ Positive Boundary

Unsigned Visual Layout:

Zero Anchor $\leftarrow [0]$ ————— $[+4,294,967,295]$ $\rightarrow$ Positive Ceiling

5. Maximum Digits Column

Identifies the exact quantity of base-10 decimal character units needed to write out the maximum possible capacity value of that type.

- `int` Max Value: `2,147,483,647` $\rightarrow$ Counting characters reveals exactly **10 digits**.
- `long long` Max Value: `9,223,372,036,854,775,807` $\rightarrow$ Counting characters reveals exactly **19 digits**.

Clarification: 19 digits references the maximum signed ceiling. If calculating the maximum unsigned value (`18,446,744,073,709,551,615`), the digit count goes up to 20 digits.

Comprehensive Operational Trace Matrix By Type

Type: `char`

- Size metrics: 1 Byte = 8 Bits
- Combinatorial configurations: $2^8 = 256$ states
- Signed limitations: -128 to 127
- Unsigned limitations: 0 to 255

- Max decimal digit depth: 3 digits

Type: `short`

- Size metrics: 2 Bytes = 16 Bits
- Combinatorial configurations: $2^{16} = 65,536$ states
- Signed limitations: -32,768 to 32,767
- Unsigned limitations: 0 to 65,535
- Max decimal digit depth: 5 digits

Type: `int`

- Size metrics: 4 Bytes = 32 Bits
- Combinatorial configurations: $2^{32} = 4,294,967,296$ states
- Signed limitations: -2,147,483,648 to 2,147,483,647
- Unsigned limitations: 0 to 4,294,967,295
- Max decimal digit depth: 10 digits

Type: `long long`

- Size metrics: 8 Bytes = 64 Bits
- Combinatorial configurations: $2^{64} = 18,446,744,073,709,551,616$ states
- Signed limitations: -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
- Unsigned limitations: 0 to 18,446,744,073,709,551,615
- Max decimal digit depth: 19 digits (signed) / 20 digits (unsigned)

Core Competitive Programming Takeaway Summary

Instead of rote-memorizing huge numbers, always remember the mathematical foundations:

- Unsigned n -bit limits range from: $0 \text{ ext{ to } } 2^n - 1$
- Signed n -bit limits range from: $-2^{n-1} \text{ ext{ to } } 2^{n-1} - 1$

Using these structural formulas allows you to derive any type constraint systematically on the fly.